

中科大哨兵2025技术报告

张昊鹏 邵宜阳 孙能晶 唐煜程 刘彦宏 涂成浪

RoboWalker2025

中国科学技术大学

1 前言和系统拓扑

本文档由RoboWalker战队编写，描述RMUC/RMUL2025中哨兵导航的方案设计和技术路线。感谢RoboWalker战队成员们的支持和努力！本文档主要编写者为：**张昊鹏/邵宜阳**。

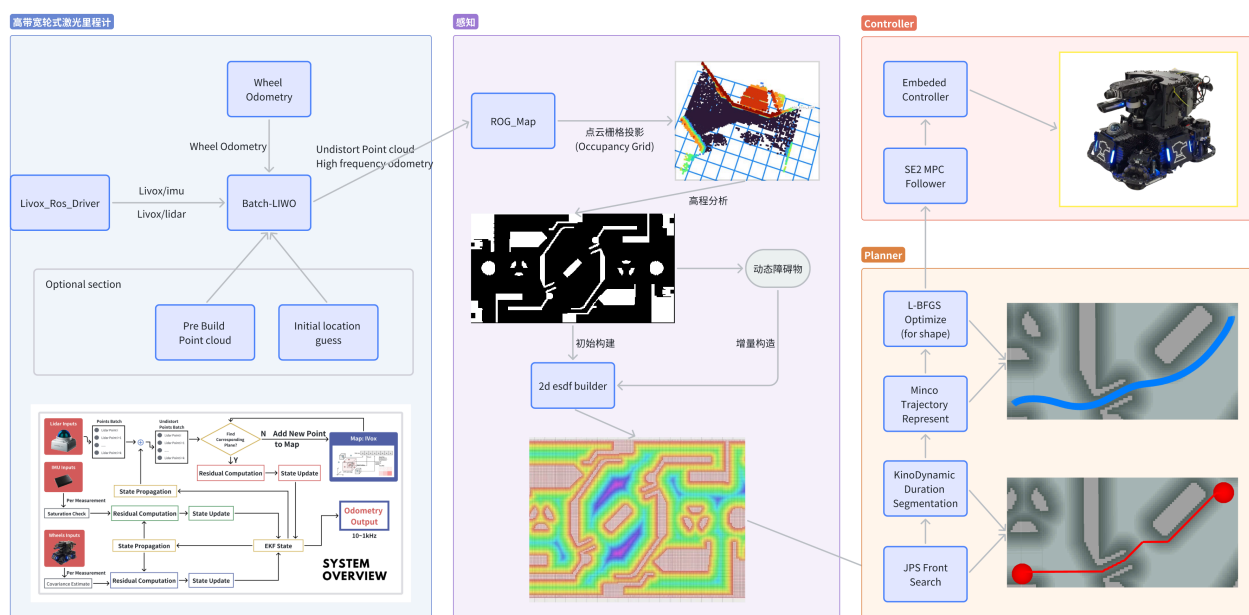
我们的主干系统基于ROS2进行开发，所以一些ROS2相关的我们认为很重要的调优经验也会出现在该文档之中。算是一种“工作量证明”。

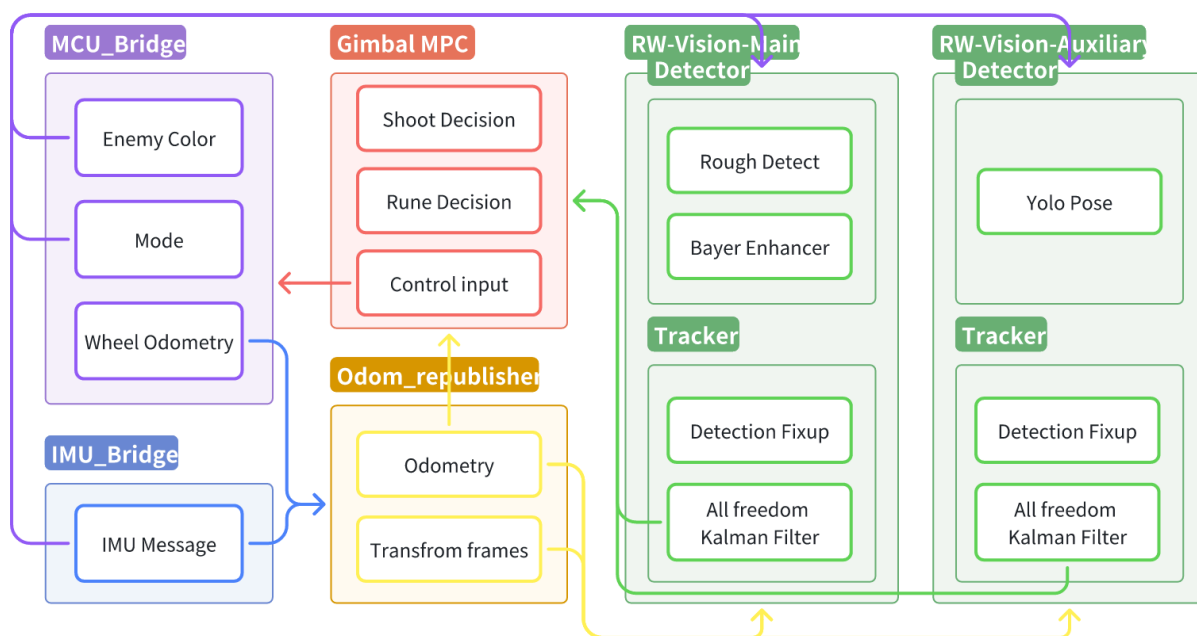
导航系统的每一个模块都在我们的理解下进行实现，每一个模块都蕴含着我们的思考和努力。失败经验将尽可能不提及以降低文档长度，如果有兴趣聊聊具体的心路历程可以联系RoboWalker主要文档编写者进行讨论。导航和业界更加相关，希望我们的导航系统对于业界能够有所启发，出于这个目的，我们会从里程计，感知，路径规划最后到控制器尽可能完全开源并附上详细文档。这部分内容需要时间实现，所以我们将在之后逐步完成，敬请期待。

视觉系统在本文档中只讨论技术路线选择，我们的自瞄发源于开源的RM Vision，出于我们对于简洁和通用的追求进行了重构，我们在此提供我们的设计思路和性能指标供大家参考。本文档中会提供视觉系统的改进思路和一些性能指标，毫不夸张的说，RW Vision是我们比起导航系统更为满意的作品。

下面我们将基于各个技术目标分别阐述每个技术部分的设计思路和实现方式。下面的讨论主要基于技术路线选择和实现细节讨论（“为什么”和“怎么做”），默认“是什么”大家都已了解，不过多进行介绍（给出参考工作的链接给大家参考），同时参考工作中已经呈现的公式和算法流程不做过多介绍。如有感兴趣/不太理解部分和指导意见可联系RoboWalker相关队员进行讨论或在RoboMaster BBS进行留言（我们推荐后者），期待您的指正。

这里我们给出本赛季导航系统和自瞄系统的模块关系拓扑：





2 赛季改进路线

我们给出赛季的主要改进路线并基于内容这些进行讨论

2.1 机械

- 两舵两全向，尽可能窄，雷达和底盘固连，大小云台结构，高结构刚度

2.2 嵌入式系统

- 全车力控，主动力矩分配，鲁棒速度估计器，功率控制

2.3 算法

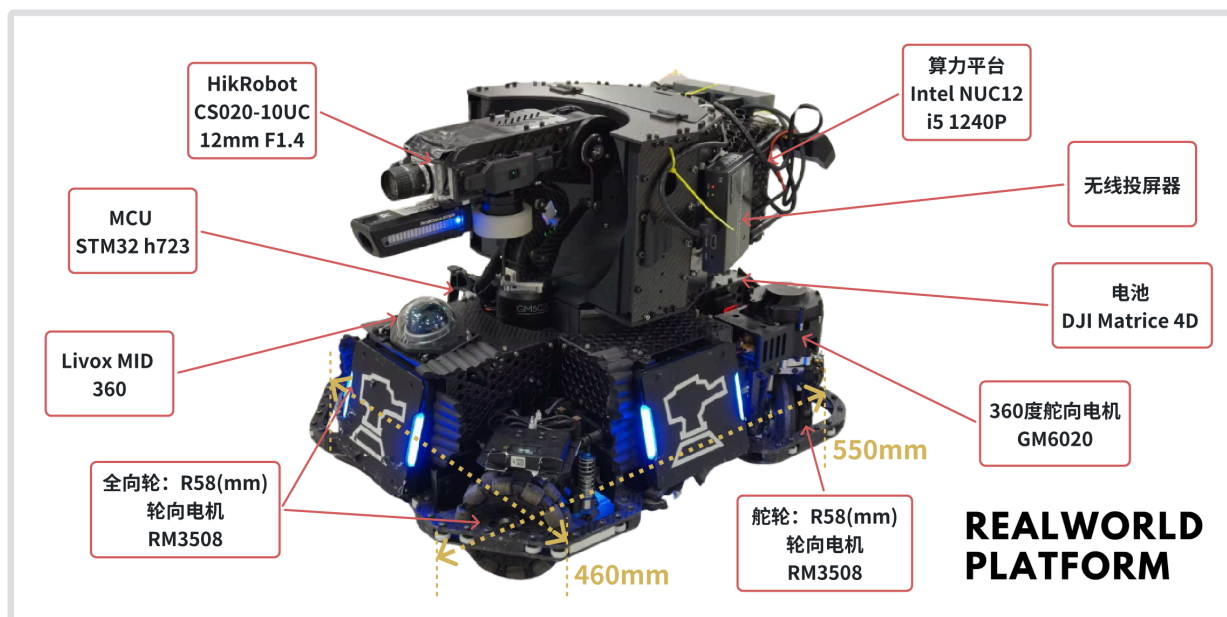
- 导航：高频鲁棒里程计，高频鲁棒感知方案，高效轨迹表示和优化方案，控制方案设计
- 自瞄：高性能陀螺仪设计，鲁棒识别前端，高性能观测器，高性能控制器

3 机械

机械部分主要设计者和实现者为**孙能晶**同学，他带着**刘博睿**同学以很高的效率完成了从设计到组装到改进再到维护的所有工作。我们在这里不太多讨论机械的指标，后续可能会开源机械结构。我们在这里给出指标的示例图片供大家参考。特别说明我们在国赛时主摄使用CS016-10UC+5mm F1.4，辅摄使用CS020-10UC+6mm，算力平台更换为NUC Ultra7-155h。

机械设计上主要参考了RUMC2024哈尔滨工业大学的哨兵实现，融合了我们对于过隧道需求的思考：

- 两舵轮两全向轮的后驱车：**减少打滑**，提供**稳定轮速计**，高上坡能力，在一定定位误差下可以“**撞进**”隧道
- 大小云台方案：高载弹量，高云台响应能力，高一致性自瞄时电机转速基准（不同转速下电机响应能力不同）
- 雷达置于底盘上：**高雷达固定刚度**，**稳定的位姿差**，尽可能**大视野范围**



4 嵌入式系统

这部分内容由刘彦宏同学实现，他以很高的效率完成了工作并在比赛前完成了科目二和科目三考试（可惜科三没过）。他是数学系的，所以符号定义比较严格，我的毕业论文曾经被他严肃阅读并指出了符号错误。这部分设计主要介绍我们在底盘解算和控制上的心得，我们后续还对这部分进行了许多改进，本文档提供完整形态考核技术报告的版本，算是早期版本。详细内容将与MPC一同在后续开源。特别值得一提的是，我们的哨兵没有安装超级电容，底盘在大约85w的最大功率下运行，参考国赛速度可以说明嵌入式实现还是不错的。

4.1 系统架构

哨兵机器人嵌入式软件系统架构分为6层。下面从顶层到底层分别介绍。

- **应用层**：包括CAN中断回调函数、USB回调函数、UART回调函数、定时器回调函数、主循环的具体实现。
- **功能模块层**：用类的方式实现各个功能模块的参数和成员函数。包括底盘、云台和发射机构。底盘包括轮向力矩解算和舵向角度解算，云台包括Pitch以及大小两个Yaw电机的PID，发射机构包括摩擦轮以及拨弹盘电机的解算。
- **硬件驱动层**：用类的方式实现各个硬件的驱动。包括达妙h7开发板板载资源的驱动程序、M3508电机的驱动程序、GM6020电机驱动程序、翎控电机驱动程序、遥控器驱动程序。包含发送命令、接收反馈、数据处理、PID计算等功能。
- **中间件层**：包含PID算法的功能包（类）以及自定义数学运算（如矩阵运算、限幅函数、大小端转换等）。
- **硬件抽象层**：对HAL库中CAN、UART等单片机外设库的重新封装。
- **HAL库**：由stm32cubeMX生成的开发者库。

哨兵机器人的嵌入式库函数由stm32cubeMX生成，使用armgcc+cmake+make作为交叉编译工具链，使用Clion作为IDE来开发，提高代码补全、提示和警告的完整性，提高编译速度，从而提高开发效率。

4.2 底盘解算

哨兵机器人的底盘采用**力控方案**，大致的控制流程为：

1. 用期望的速度和角速度进行速度解算，以此获得舵向的期望方向和轮向的期望速度
2. 从当前轮速解算出车体的速度和角速度
3. 通过PID计算得到期望的力和力矩

5. 进行功率控制

4.2.1 记号约定

在这部分的文档前, 我们先对下文涉及到的记号进行约定. 首先我们约定, 底盘拥有两个全向轮的一边为底盘的前侧, 对应为 $+x$ 方向, 以此在底盘平面正交于 x 轴的方向规定 y 轴, 利用右手系规定 z 轴, 使得 $+z$ 方向为向上 (也即底盘安装云台的方向).

记号	描述
F_x^c	底盘 $+x$ 方向的力
F_y^c	底盘 $+y$ 方向的力
T^c	底盘 $+z$ 方向的力矩
v_x^c	底盘 $+x$ 方向的速度
v_y^c	底盘 $+y$ 方向的速度
ω^c	底盘 $+z$ 方向的角速度
F_1	位于第一象限的全向轮提供的力, 以含 $+x$ 方向分量的一侧为正
v_1	位于第一象限的全向轮的速度, 正方向与 F_1 正方向一致
F_2, δ_2	位于第二象限的舵轮提供的力, 此力与 $+x$ 的夹角为 δ_2
v_2	位于第二象限的舵轮的速度, 正方向与 F_2 正方向一致
F_3, δ_3	位于第三象限的舵轮提供的力, 此力与 $+x$ 的夹角为 δ_3
v_3	位于第三象限的舵轮的速度, 正方向与 F_3 正方向一致
F_4	位于第四象限全向轮提供的力, 以含 $+x$ 方向分量的一侧为正
v_4	位于第四象限的全向轮的速度, 正方向与 F_4 正方向一致
a	底盘沿 x 方向长度的一半
b	底盘沿 y 方向长度的一半
c	$\sqrt{a^2 + b^2}$
β	$\arccos \frac{a}{c}$

4.2.2 正向运动学解算

容易的我们有以下的关系:

$$\begin{pmatrix} 1 & & & & \\ & \cos \delta_2 & & & \\ & \sin \delta_2 & & & \\ & & \cos \delta_3 & & \\ & & \sin \delta_3 & & \\ & & & & 1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix} = \begin{pmatrix} \sin \beta & -\cos \beta & -c \\ 1 & & -b \\ & 1 & -a \\ 1 & & b \\ & 1 & -a \\ \sin \beta & \cos \beta & c \end{pmatrix} \begin{pmatrix} v_x^c \\ v_y^c \\ \omega^c \end{pmatrix}$$

将左侧矩阵记为 $\mathbf{A}$, 右侧的矩阵记为 $\mathbf{B}$.

因为 $\mathbf{A}$ 非常的简单, 因此从右侧出发, 我们可以十分容易的求解出左侧的未知量 $v_1, v_2, v_3, v_4, \delta_2, \delta_3$, 这也就完成了正向运动学解算.

4.2.3 逆向运动学解算

我们依旧从 (1) 出发, 来进行逆向运动学解算, 这实际上也就是需要解决一个超定的线性方程组. 一种简单的处理方法是直接使用 Moore-Penrose 逆来直接获得一个最小二乘解. 因为 $\mathbf{B}$ 是一个常数矩阵, 计算 Moore-Penrose 逆的过程可以提前处理, 不会给单片机带来计算负担. 这样做最大的优点是简单, 相应速度快. 但是相应的会带来一些问题: 首先是由于噪声等的影响, 这样获得的结果并不光滑, 可能对于力控要计算的 PID 不太友好; 其次, 这个方法难以处理一些特殊情况, 比如在实际测试中我们发现某些轮子更容易打滑, 这样以来这个轮子提供的信息就是不可靠的, 进而我们也就希望尽可能少的使用打滑轮子的信息. 为了解决这些问题, 在实际操作中我们使用了 Kalman 滤波器来进行逆向运动学解算. 这种方法使得解算结果更平滑, 二来可以通过加入权重来调整解算方法来避免误差.

离散时间的一般的卡尔曼滤波可以写为:

$$\begin{aligned}\mathbf{x}_k^- &= \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k \\ \mathbf{P}_k^- &= \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^T + \mathbf{Q} \\ \mathbf{K}_k &= \mathbf{P}_k^- \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^T + \mathbf{R})^{-1} \\ \mathbf{x}_k &= \mathbf{x}_k^- + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}_k \mathbf{x}_k^-) \\ \mathbf{P}_k &= (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^-\end{aligned}$$

我们对卡尔曼滤波器的输入应该是四个轮子上的速度, 需要观测的量是车体中心的速度和角速度. 因此我们有:

$$\mathbf{x} = \begin{pmatrix} v_x^c \\ v_y^c \\ \omega^c \end{pmatrix}$$

$$\mathbf{z} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix}$$

由于不方便获得车体的加速度, 我们做车体中心速度和角速度恒定的假设, 也即:

$$\mathbf{u}_k = \mathbf{0}$$

应在惯性系统下考虑速度. 假设前一个时钟周期的真值状态已知且时间间隔已知, 则可以很容易地获得与前一个时钟周期的角度差, 从而可以通过二维旋转获得当前时钟周期的底盘速度, 从而我们有:

$$\mathbf{F} = \begin{pmatrix} \cos(\omega dt) & \sin(\omega dt) & \\ -\sin(\omega dt) & \cos(\omega dt) & \\ & & 1 \end{pmatrix}$$

从 (1) 出发, 我们利用 $\mathbf{A}^+ \mathbf{B}$ 作为我们的转移矩阵 (此处 $\mathbf{A}^+$ 指的是 $\mathbf{A}$ 的 Moore-Penrose 广义逆), 也就有:

$$\mathbf{H} = \mathbf{A}^+ \mathbf{B} = \begin{pmatrix} \sin \beta & -\cos \beta & c \\ \cos \delta_2 & \sin \delta_2 & -b \cos \delta_2 - a \sin \delta_2 \\ \cos \delta_3 & \sin \delta_3 & b \cos \delta_3 - a \sin \delta_3 \\ \sin \beta & \cos \beta & c \end{pmatrix}$$

矩阵 $\mathbf{Q}$ 和矩阵 $\mathbf{R}$ 的设定是此处解算比较关键的一点. 可以先给定其中的一个, 根据效果调整另外一个. 在没有设备进行参考速度和角速度的获取时, 可以把通过 Moore-Penrose 广义逆进行的逆向运动学解算结果作为参考, 观察滤波器的滞后表现等.

4.2.4 力分配解算

容易的我们有以下的关系:

$$\begin{pmatrix} \frac{b}{c} & \cos \delta_2 & \cos \delta_3 & \frac{b}{c} \\ -\frac{a}{c} & \sin \delta_2 & \sin \delta_3 & \frac{a}{c} \\ -c & -c \sin(\beta + \delta_2) & c \sin(\beta - \delta_3) & c \end{pmatrix} \begin{pmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{pmatrix} = \begin{pmatrix} F_x^c \\ F_y^c \\ T^c \end{pmatrix}$$

将方程记为 $\mathbf{C}\mathbf{x} = \mathbf{y}$.

在前面我们提到舵轮的舵向电机方向由速度正解算提供, 这样做的其中一个原因也在这里体现. 如果我们想通过 (12) 来同时解算轮向的力和舵向电机的方向, 那么我们将会试图从 3 个非线性的方程中求解出 6 个未知量, 这是较为困难的; 而如果我们通过 (1) 事先将舵向电机的方向确定下来, 那么我们面对的将只会是一个欠定的线性方程组, 为后续的解算带来极大的方便.

我们先来观察系数矩阵 C 的性质. 这是一个 3×4 阶矩阵, 也就有 $\text{rank } C \leq 3$; 而从物理意义上出发, 我们可以确信三个方程是独立的 (这在数学上也可以证明), 也就有 $\text{rank } C \geq 3$, 从而迫使 $\text{rank } C = 3$, 是一个行满秩矩阵. 这也意味着 (12) 的解必然是 $\mathbb{R}^4$ 上的一条直线, 在通常的情况下我们要做的就是在这条直线上选取一个合适的点.

我们可以相信使用 Moore-Penrose 广义逆求解方程得到的最小最小二乘解 (考虑到方程一定有解, 故实际上是最小解) 是一个不坏的选择, 因此我们的思路就是对这个解进行微调优化. 因为 C 是一个行满秩矩阵, 我们有:

$$C^+ = C^H(CC^H)^{-1}$$

因为这个矩阵规模很小, 所以这样做在单片机上是可行的. 这样一来我们就可以获得一个初步的力分配, 即 $x_{pre} = C^+y$.

接下来我们通过处理两种期望外的情况来进行力分配的微调优化. 这两种情况分别是 "正常" 的打滑和 "不正常" 的打滑.

"正常" 的打滑主要发生在底盘上坡时. 我们车体的质量分配决定了位置低的轮子会有更大的压力, 从而有机会提供比位置高的轮子更大的动力. 在前进上坡时表现为前侧的两个全向轮打滑, 而后侧的舵轮转不动. 处理这种情况, 我们只需要先通过高斯消元求解出 $\text{Ker } C$ (请注意因为 $\text{rank } C = 3$, 有 $\dim \text{Ker } C = 1$, 此处只需要求解出 $\text{Ker } C$ 中的一个非零向量即可), 再把这个空间中的某个向量加到我们的解上. 一种简单的处理方法如下:

- 准备一个变量 cof , 初值为 0, 作为即将使用的系数
- 每次求解出 $\text{Ker } C$ 中的非零向量后, 将其单位化并且使得其第一个分量为正, 记为 k
- 将新得到的向量与上一个时钟周期得到的向量做内积, 也即获取他们夹角的余弦. 如果余弦值过小, 需要将 cof 置为 0
- 将每个轮子的转速和期望的转速做比值, 找到相对转速最低的轮子, 认为这个轮子需要更大的力矩
- 计算 cof , k 对应分量, x_{pre} 对应分量的乘积, 如果为正那么保持正负的增大 cof 的绝对值, 否则减小
- 将 x_{pre} 与 $\text{cof} \cdot \|x_{pre}\| \cdot k$ 相加作为新的力分配, 记为 x , 这里必须保证 x 和 x_{pre} 的每个分量的正负一致, 否则减小 cof 的绝对值

"不正常" 的打滑主要发生在存在轮子悬空时, 具体表现是某一轮子的相对转速远大于其他轮子. 这时只需要将该轮的力矩置零, 直到其相对转速回到合理的位置后再恢复力矩输出即可.

4.2.5 功率控制

对于单个电机, 我们建立功率模型:

$$P = k_1\omega^2 + k_2I^2 + k_3I\omega + k_4|\omega|$$

具体的系数可以通过采集数据后进行最小二乘拟合得到.

我们的功率控制策略是:

1. 估计舵向电机功率, 观察是否超过人为划定的舵向电机最大功率, 如果超过, 则计算功率因子, 分别设定两个舵向电机的电流使得其估计功率变为原先的功率因子倍 (通过求解二次方程解决);
2. 计算可以分给轮向电机的功率;
3. 估计轮向电机功率, 观察其是否超过可用功率; 如果超过, 则计算电流因子, 为所有的轮向电机电流乘上同一个电流因子使得总的估计功率为可以分配给轮向电机的功率, 这只需要求解一个系数比较复杂的二次方程.

这样的策略最大的好处是不会破坏我们精心计算的力分配解算结果.

功率控制是我们的底盘控制流程的最后一步, 但在将我们的结果赋值给电机前, 我们应该注意我们的结果有没有超过电机的电流上限 (对于我们使用的 M3508 电机而言, 20A), 如果超过, 我们要为所有轮向电机电流乘上同一个系数, 使得电流不超限.

5 导航算法

导航算法的里程计，感知和规划部分由张昊鹏实现，控制器由邵宜阳实现。

里程计算法Batch-LIWO在24年第四季度进行调研和开发，在25年端午节获得重大提升，并最终作为张昊鹏本科毕设的编写背景。

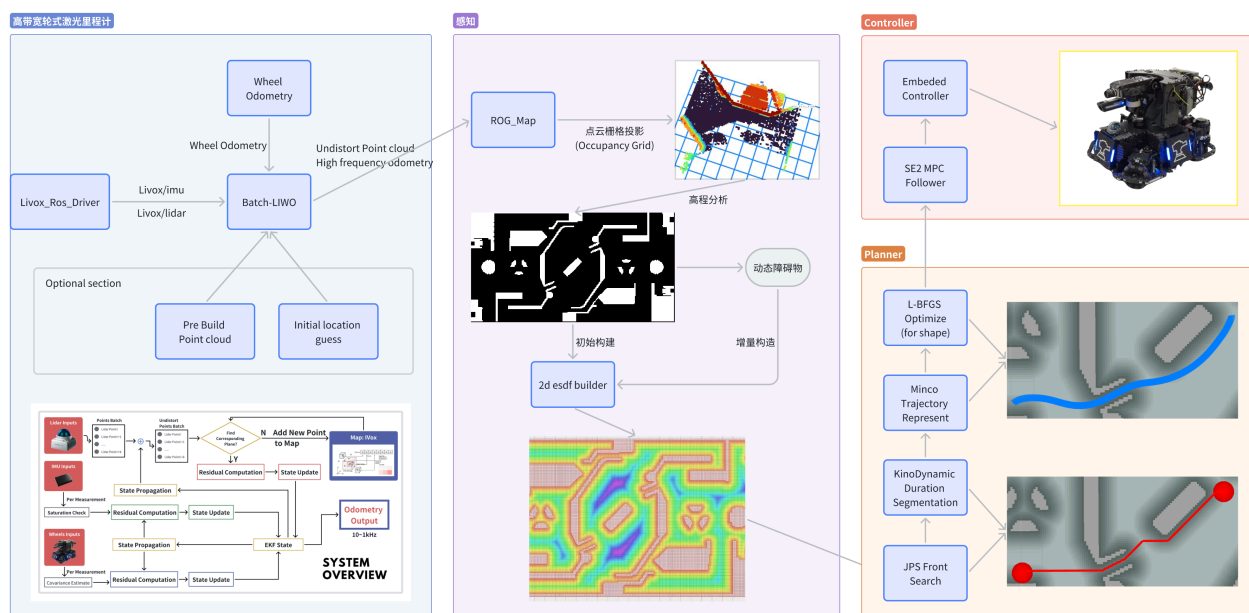
感知模块在24年下半年集中进行开发，尝试了基本所有非网络实现的无预处理方法（我们希望它可以有更好的适用性），最终选择了本文中后续会讨论的简单实现方案。感知的前端使用了[hku-mars/ROG-Map](https://github.com/hku-mars/ROG-Map)，这个项目主体实现了以机器人为中心的Occupancy Grid构建。其代码编写规范，实现清晰流畅，我们在它的基础上将ROG-MAP添加了并行化实现和对许久没看到物体的遗忘(fading)并迁移到ROS2(后续不独立讨论，我们所有模块都基于ROS2进行通信和调试)。

规划算法使用了二维ESDF作为感知前端，通过感知模块的输出和先验二维地图(可选)构造ESDF。轨迹表示和优化方案采用了Minco作为轨迹表示方法，L-BFGS作为优化器。同时我们针对狭窄地形的梯度震荡问题进行了思考和处理，给出了二次插值方案和两次优化的策略得到了合理的，包含丰富动力学信息的轨迹并大大提高了计算速度(80ms->10ms)和规划稳定性。同时我们设计了适合Minco表示重规划策略，实现了高频率重规划并保证了重规划稳定性（运动状态不突变）。

控制算法上我们使用了SE2 MPC作为控制器，该控制器不止可以控制机器人的平移，还可以控制机器人的底盘跟随。我们每个模块都充分相信上游模块的性能，MPC模块的设计目标为让机器人牢牢贴合规划出的轨迹。我们针对这个问题设计了沿轨迹方向和垂直轨迹方向的两个不同规划器行为。同时控制算法接收规划算法中对控制点附件空间占据情况的描述来控制底盘是否需要强行进入跟随路径朝向模式。

总体上而言我们以较少的模块数和代码量实现了从前端里程计到最后后端控制器的设计，实现了稳定运行和通过狭窄地形的功能。同时因为架构平坦简单，大大加速了调试和开发过程，使得我们有更多时间重复进行机器人布线（bushi）。

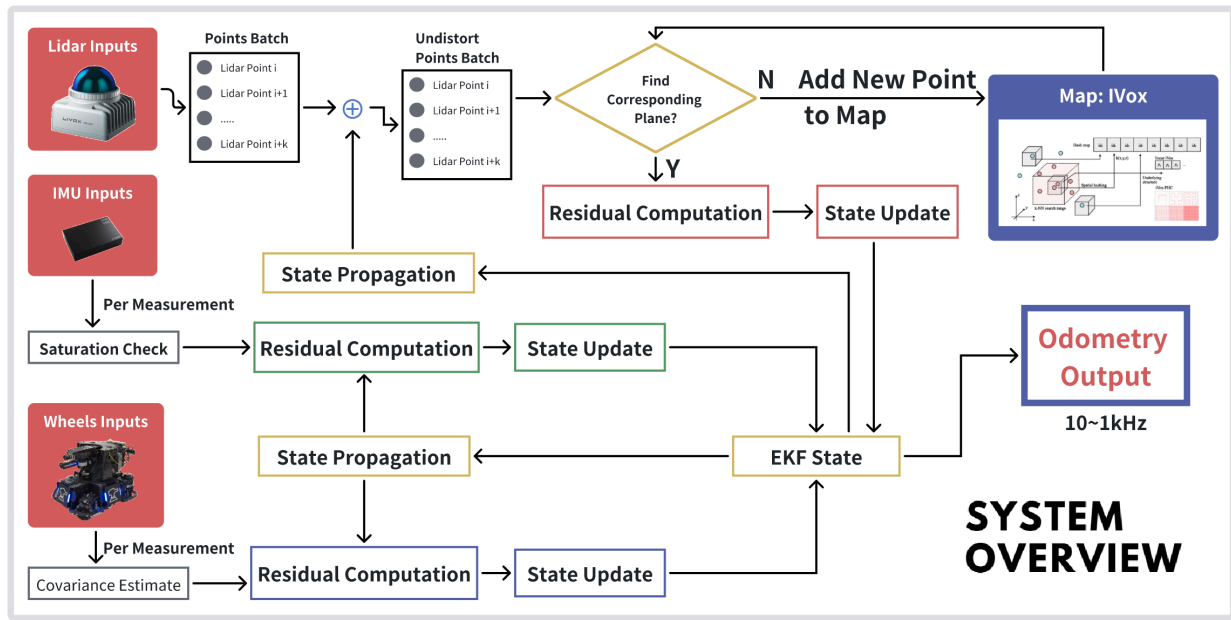
这部分文档在技术报告的基础上进行了修改，后续肯定会开源技术报告（目前还没发现怎么开源），感兴趣可以对照着看。这份文档中我们只讨论和最后的实现相关的内容。



5.1 Batch-LIWO

Batch-LIWO总体上基于**Point-LIO**方案进行改进，吸纳Point-LIO的优点，在I-ESKF框架的基础上融合了轮速里程计，给出了轮速里程计的置信度量方案，同时实现了基于Batch的更新方案使得系统拥有更大的点云吞吐量从和伴随其而来的更高里程计精度。实现了在降低运行负载的基础上**提高了撞击/高转速/点云缺失的狭窄隧道等极端条件下的里程计稳定性**。

这部分的实现我们只做意识流上的讨论，具体内容我们给出参考文献（恐龙的毕业论文2333，写得比较随意还请见谅）。



5.1.1 系统设计

状态量维度为 $SO(3)+R^{21}$ ， R^{21} 的定义在下面列出。采用了和Point-LIO相同的把IMU的角速度和加速度的方案进行实现（这是Point Lio彻底区别与其它Lio算法的根源所在）。将角速度写入状态量，这么做的优点在于可以通过测量噪声很自然地描述IMU的噪声和加速度计或陀螺仪饱和后可以通过放弃通过IMU更新，仅通过雷达点更新来保证系统稳定性。

- 世界系下IMU的位置
- 世界系下IMU的速度
- IMU的陀螺仪偏置
- IMU的加速度偏置
- 重力
- IMU的角速度
- IMU的加速度

5.1.2 基于Batch的更新策略

Point-Lio为Point Wise更新，将每个点作为一次测量，最理想情况下把MID360激光雷达变成了200000Hz的传感器（和点云数量相同）。这个频率远高于IMU频率（200Hz）。所以通过单个雷达点计算残差后低置信度更新状态量，然后在IMU观测到来的时间高置信度更新状态量是很合理的。正是因为如此，Point-Lio有比Fast-Lio更好的极端动态下的里程计稳定性。但是，这样的方案的问题在于系统会变得脆弱：

- Point Wise的时序更新，所以难以进行大规模并行化处理，如何选择有代表性的关键点很重要
- 为了避免Point Lio运行时间超过预期，控制输入点数目很重要

基于这些痛点，我们设计了基于Batch的Update策略。我们通过状态量将一个以时间打包的Batch（通常为1ms）内的所有点云变换到相同的时间戳之下。对每个点单独计算残差，然后将残差，测量矩阵进行拼接后进行Update操作。这样相当于每个测量点以相同的置信度进行更新。同时因为减少了Update和Predict次数，系统运行效率得到了很大提高。

因为Mid360的非重复扫描特性，雷达可以在很短时间内扫码空间中的多个平面（4 Laser Lines）。那么每次的Batch中将包含多个平面的信息，有效约束了自由度，降低了每次Update的自由度更新不全的问题，提高了系统稳定性和精度。

5.1.3 轮速融合

观测退化场景下LIO系统难以保证稳定，所以我们基于I-ESKF框架融合了轮速，一个真正高带宽而且无累积误差的测量来源（MID360内置IMU 4g饱和+二次积分得到位移容易发散）。

所以我们将轮速加入了ESKF之中，ESKF的融合框架下我们只需要给出残差和协方差描述。残差定义为IMU坐标系下的残差，协方差为通过轮速期望残差的倍数加上一个基准噪声作为轮速测量的协方差。这个期望残差的度量方式为：

1. 获得当前时刻的轮速和舵轮角度，通过最小二乘法进行求解得到最小二乘意义下的当前时刻底盘速度反馈（无打滑假设）
2. 将最小二乘解的底盘速度通过舵轮角度投影回四个轮的期望轮速，将期望轮速和实际轮速进行权重相同的加权平方和得到期望残差。

这个度量方式是很保守的，在一个轮子失效的时候会很大程度降低轮速的置信度，这对于多种观测源组成的系统而言是好事。我们允许轮速观测源短暂被误判为失效（还有IMU和点云在工作），但无法匀速其对速度的错误更新。同样的，我们对IMU和点云融合也是作此要求。

5.1.4 工程实现

我们参考Point-LIO进行代码实现并重新实现了里程计的许多模块，包括：传感器队列，独立处理线程，主处理逻辑。目标是简化代码逻辑，提高稳定性和运行效率。我们将在

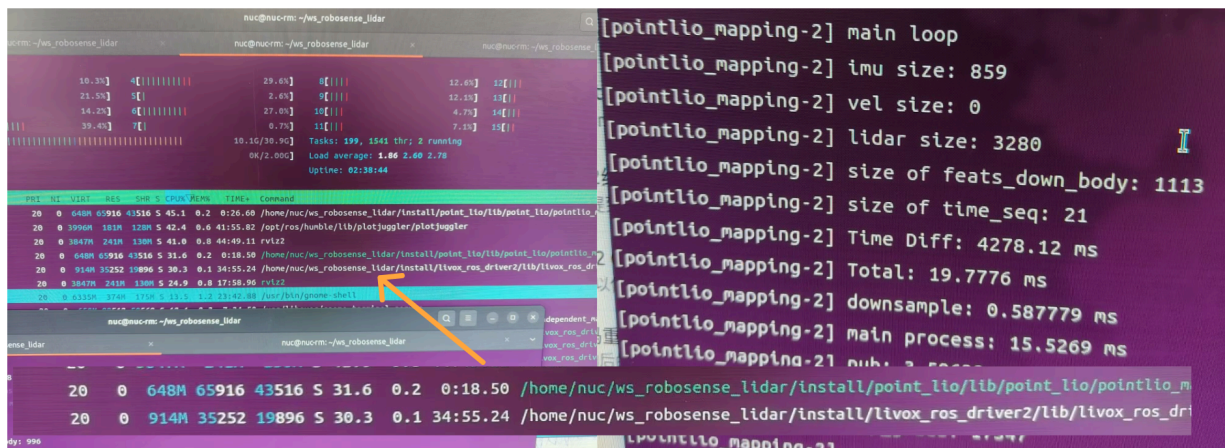
[ZhangHaopeng-Dino/Batch-LIWO](#)项目下完成这个实现的开源。

我们在赛季中测试了RoboSense Airy，它的数据包更大，对系统造成了很大负担。于是我们实现了通过共享内存进行传输的启动脚本shm_lio_bringup和RoboSense-Driver，同样的，我们也修改了Livox-ROS-Drivers，目的是降低系统负载，提高效率。我们最后使用了Mid360作为车载雷达，非重复扫描对于里程计场景而言无比正确，没有平替。

同时为了后端模块对点云密度的需求，我们给Batch-LIWO添加了完整点云输出。将被降采样舍弃的点云通过一个队列进行时间升序排列，每次update获取新的状态量之后进行一小步前向预积分，对齐点云时间戳后将点云变换到世界坐标系下。

下面我们给出我们测试出的性能指标（详细开源中会附带更多测试）：

- intel NUC 1240p，单线程，处理6w pts/s（不包括降采样舍弃的），占用30-40%资源
- 20小时测试无内存泄露和CPU负载上升，系统延迟不变
- 14rad小陀螺稳定，3m/s冲撞稳定



5.2 全局定位

有许多好奇的宝宝问我们全局定位是怎么做的，在这里特别说一下我们没有特别实现全局定位模块。我们通过将预建图得到的点云和姿态提供给Batch-LIWO来实现全局定位的功能（预先构造iVox和直接写入初始位姿xyz+yaw）。每次要把机器人摆在基本相同位置启动，我们摆放在大概有20cm位置，30度以内yaw误差的情况下可以稳定启动（国赛上场测试结果）。我们的Batch-LIWO在整个国赛过程中运行稳定，基本可以说是没有出过问题（唯一问题是底盘掉电后反馈了错误的高置信度轮速，导致里程计漂移，打RPS的时候发现死了就往奇怪地方跑2333，打完RPS已经严肃改正，后续无明显问题）。

为什么不采用基于局部迭代的方案，其实Lio的实现本来就基于类似ICP的思路，再加一个简单的ICP不会有任何提升。但是加上随机采样的情况下，系统负载和模块间的时间同步将变得不稳定，可能是对系统更大的伤害。因为稳定不跳变的里程计是后续所有模块的基础。所以我们的思路是强化里程计实现无需重定位的鲁棒位置估计效果。

5.3 感知方案

5.3.1 基于3D Occupancy Grid的点云结构化方案

5.3.1.1 方案概述

总体上而言感知方案基于三维Occupancy Grid实现。以3D RoboCenter 的Sliding Map实现（**ROG-MAP**）作为基底，将系统迁移到ROS2上，添加了**Fading**（长时间未更新障碍点遗忘）和Raycasting的**并行化处理**。实现了2.5cm分辨率下10m * 10m * 1m地图大小的MID360完整点云(遗弃无效遮挡点后大约8w/s)下的30%时间占用运行（50hz下的大约帧时间6ms）。

实现上主要基于HKU MARS开源的ROG-MAP进行了移植和修改(<https://github.com/hku-mars/ROG-Map>): An Efficient Robocentric Occupancy Grid Map for Large-scene and High-resolution LiDAR-based Motion Planning。

定运行。

5.4 障碍物识别方案

5.4.1 方案概述

基于ROG-Map输出的稠密结构化点云，我们尝试了许多方案进行可通行区域分割和障碍物识别。最终选择了基于3D Occupancy Grid降噪后的稳定结构化点云通过高程分析的手段获得障碍物。对障碍物基于时间和位置聚类后通过卡尔曼滤波器估计障碍物的运动速度，获得动态障碍物和基本静态障碍物（这部分没有上场也没有太多测试）。

5.4.2 可通行区域分割

5.4.2.1 传统方案

基于偏传统的方法，比如PatchWork++，LineFit Seg，PUTN和其他一些基于特征分类的方案。我们进行了大量测试和理解，没有一个能满足我们对于实时性和分割可靠性的要求（因为一个点分割错误就可能让机器人不选择通过隧道）。

最好的传统算法方案如下的手搓方案

1. 将3D结构化点云转换为两张二维灰度图：最低高度，最高高度（最高的判断标准为往上找机器人高度都是空闲则为最高）
2. 作差后获得高度差图，进行简单图片处理减噪。
3. 将高度差图计算3*3方差
4. 根据方差阈值进行分割

这样的方案有几个问题：

1. 前端点云并不完全稠密，Raycasting方案有不少空缺。同时因为是实时建图，这会导致高度差并不准确（比如障碍物只扫描到了表面）。造成误识别和错检出。
2. 系统性能表现对调参要求高，不适合rm这样的高动态要求的复杂场景。

所以我们自然放弃了这条道路。

5.4.2.2 基于RM场景的方案

我们的感知前端其实给出了很好的点云表示，它是结构化的。基于这个特点，我们直接对每个离散的x y坐标进行高程分析。根据z轴上点云的高度差和占据率进行分类。这个方案测试下来鲁棒性非常理想，效率也很高，非常适合我们的工况。

- 高度差：最高点高度减最低点高度H
- 占据率：最高点到最低点之间的点云数n，res为分辨率，占据率= $((n + 1) * res) / H$

为什么这个方案好用，其主要是因为RoboMaster场景中的墙大都是竖直的，雷达会扫描到一个平面，加上前端结构化的特点降低了面噪声的影响，使得高程分析可以稳定运行。

5.4.2.3 动态障碍物提取

这里主要讨论动态障碍物的识别和观测。

基于提出的障碍物分割方案，我们可以对判断为障碍的点进行聚类。因为基于Occupancy Map方案实现的障碍物识别一定会留下尾迹（原理性问题），比起思考怎么去除尾迹，不如将尾迹作为观测点进行考虑。记录尾迹上每个点的更新时间，对相同更新时间且彼此临近的点进行聚类，然后通过简单的基于匀速模型的卡尔曼滤波进行观测和卡方检测检测下一个聚类结果是否符合模型，如果符合则继续往下进行检测和更新。这样我们就得到了所有障碍物和他们的运动速度，精度一般，但对于纯雷达方案而言也差不多了。

5.5 基于MINCO轨迹表示的规划方案

5.5.1 方案概述

- 规划器部分首先应该考虑的是**轨迹表示和优化方案**，这部分是影响性能的最大头。目前的对计算性能要求较低的非采样轨迹以B-Spline轨迹和Minco轨迹为主。**Minco轨迹好在方案是经过控制点的，可以直接通过控制点来控制轨迹**。这个特性是非常大的优势，在后面规划方案的每个部分都会提到这个优势。同时因为Minco比较“小众”，符合大学生的神秘的非从众心理（bushi）。最终选择了**使用基于Minco的轨迹表示方案和基于L-BFGS的无约束优化方案**，基于G-Copter项目主干进行修改。
- 然后是**障碍物梯度表示方案**，这部分选择不多，在场地场景下且能高效提供梯度的无非是1. Grid Map膨胀后构造安全走廊 2. ESDF。我们**权衡后选择了基于ESDF方案**（Euclidean Signed Distance Functions），它的优势将在后续部分中详细讨论。这个选择充满潜力但有很多工程实现上的细节。
- 接下来需要考虑**前端路径搜索算法**，我们选择了JPS。然后经过**基于梯形加减速的时间重新分配方案**重新采样轨迹，获得均匀时间差的前端路径点。
- 再接下来是**重规划算法**，这部分我们基于Minco轨迹的特性设计了三个策略：1. 全局重规划 2. 仅优化 3. 部分重规划。同时设计了起始点目标点排出障碍物的策略，保证了目标点被占据时系统的稳定性。

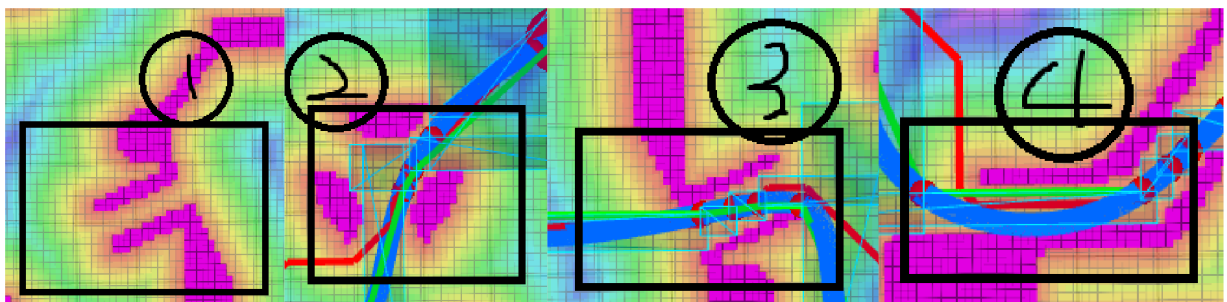
下面我们将分模块展开讨论上面提到的技术方案。

5.5.2 障碍物梯度表示

5.5.2.1 ESDF方案的优势和不足

从哈工大RMUC2024中放出的调试视频来看，他们采用了基于膨胀地图的方案，在膨胀地图上构造矩形安全走廊。我们在尝试之后发现有三个主要问题（哈工大老哥表示会开源，非常想看看他们是怎么做的，我觉得我没弄对）：

- 今年地图中**隧道是斜的，体素化存在不小误差**（参考下图1和3,4, 3的位置比较宽因为认为擦除了靠近过道障碍点。1和4通道直径应该相同但是不同）。
- 今年地图中的**隧道偏长且存在转角**，不适合走廊构造方案（参考下图4）。
- 安全走廊**无法优雅高效解决动态障碍物问题**（不能每个预测时间片都重新计算走廊），采用基于CBF的方案无法自然地和安全走廊结合。
- 因为**避障是软约束**，安全走廊只在出走廊后给予梯度，这会导致狭窄地形优化不理想，容易优化出在障碍物里的路径特别是面对进出前后都需要转弯的情况（参考下图2,3,4）。



所以我们采用基于ESDF的方案，ESDF方案从解决上面三个问题的角度来说优势是明显的：

- 根据场地地图预构造的**ESDF可以很精细**，不过是内存占用区别而已，**对实时处理时间没有影响**。（实际上场采样5cm分辨率，挺粗糙的其实）。
- 受到RC-ESDF思路的启发，我们可以将**每个运动障碍物的ESDF独立出来**（RC-ESDF把机器人自身考虑为ESDF图，但是并不适合狭窄地形，局部性太强会导致强烈震荡）。系统在每个时间点预测障碍物运动到的位置，然后比较场地ESDF和每个运动障碍物对应处的ESDF，取最小的作为ESDF值。这样的操作也不会改变插值

案，同时流程上自然流畅。（还没试，需要和前面提到的聚类方法一起使用）。

3. 可以通过设置ESDF梯度介入优化的半径范围来灵活控制与障碍物之间的距离，避免狭窄地区的优化不理想。

如果它没有劣势我们会很开心，可是事情并不总是顺利，我们在开发的过程中发现了基于ESDF方案存在下面的问题：

1. **时间调整很不灵活**：走廊方案在前后走廊交界处都可以自由调整，调整范围其实很大，对时间预分配要求很低。所以优化稳定性远好于ESDF方案。
2. **梯度震荡强烈**：因为双线性插值获得梯度的原因，峡谷形障碍物中间位置可能没有梯度（后称为梯度无效化：两个Cell到障碍物距离相同的情况），这个问题而且很常见。
3. **优化器不收敛**：震荡强烈容易超出优化器Line Search的上限，无法得到理想的满足连续性和动力学约束的结果。

5.5.2.2 针对ESDF方案不足的改进

上述三个问题对系统的影响是递增的，我们尝试解决了上面三个问题，并且获得了初步（基本能用）的结果。下面是解决这些问题的思路和方案：

1. **时间调整方面**：我们对JPS的结果经过了比较细致的重新采样。
2. **梯度震荡强烈方面**：我们通过二次函数插值避免了梯度无效化问题：根据点相对于Cell中心的位置在两个方向的2*3方格中通过三个点计算二次函数，获得了平滑的距离障碍物距离。并且根据二次函数的切线斜率直接得到了平滑的梯度，基本解决了上面提到的梯度无效化问题。
3. **优化成功与否方面**：我们发现了优化失败的主要诱因，并通过一定的优化时间的增长换取了优化结果稳定（这不是最好的方案，但是可以接受，最好的方案需要重新设计优化器的优化策略）。
4. 通过两次不同优化目标进行轨迹优化。相当于重新设计了优化策略，在写完4月份技术报告以后实现。

5.5.2.3 实际部署上的经验

我们前面提到了将动态障碍物和静态障碍物分开更新ESDF。

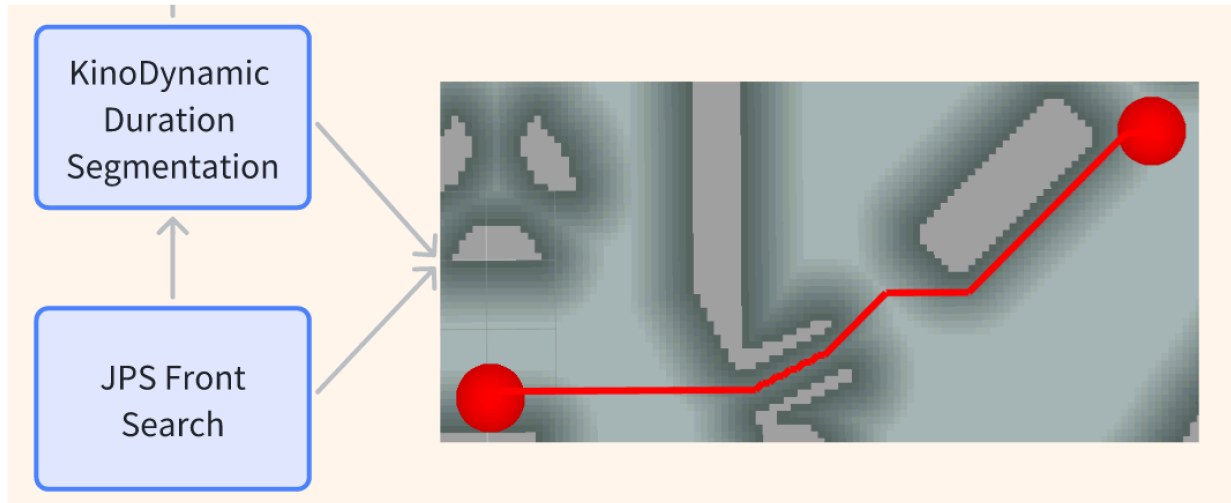
但是我们的实际测试表明，将两个障碍物合并后更新2d-ESDF在RMUC场景尺度下的0.1m分辨率下只需要2ms。在不考虑动态避障则完全可以接受。

所以我们在联盟赛版本（其实还包括分区赛和国赛）中使用了完全动态更新ESDF的方案，即每次感知模块输出的点云将用于重新构造ESDF。不到2ms的单线程构造时间并没有对系统造成什么影响，但是大大降低了编程复杂度。

我们将在RMUC2025中实装独立增量更新的ESDF模块（增量部分参考为FIESTA的实现，这赛季没有实装，但RMUC2026相信学弟会实装这个内容）。

5.5.3 前端轨迹搜索方案

参考FAST-LAB在25年开源的工作DDR-opt，这是一个二维上的基于差速模型的项目。它的前端使用了JPS进行搜索并且通过梯形加减速模型进行了时间均匀分配下的轨迹重新采样。



5.5.3.1 搜索模型的选择

为什么选择JPS: JPS 已经被证明是基于无权格子，在没有预处理的情况下寻路最快的算法

为什么不选择例如KinoDynamic A*/KinoDynamic RRT* 这样的融合运动学信息的方式: 局部性太强，难以有效利用全局信息，只适合局部的增量式地图生成下的前端轨迹搜索。

5.5.3.2 如何给JPS轨迹添加运动学信息

我们这里给出两个思路，思路二需要基于思路一实现:

思路一: 直接梯形加减速重采样

将JPS搜索结果基于以下的流程进行处理:

- 记录相邻两点间的连线长度总和 s_1 和，相邻三点间的以中间点为转角的总角度变换 s_2
- 最终距离定义为 $s = k_1 * s_1 + k_2 * s_2$ (s_1 对应直线行驶时的速度， s_2 对应旋转时的速度， k_2 一定程度上表征了轮到车辆中心的距离)
- 设置 a_{max} 和 v_{max} ，基于梯形加减速模型计算用的总时长 T
- 根据预设的期望时长 t 对轨迹进行分割并重新采样轨迹
- 整理得到新的前端轨迹采样点和每个轨迹采样点的时间

思路二: 在思路一基础上添加一次放松条件的Minco轨迹优化

思路一这样的简单实现存在问题: JPS搜索得到的轨迹不平滑，简单的基于思路一的重采样会造成时间分配不合理（折角处明显变慢）。这会直接导致后端优化失败（结果扭成麻花形状/轨迹时间特别长速度特别慢）。

这些问题可以通过调参来部分避免，但在一些想不到的时候问题仍会发生。这导致规划模块向外输出的结果不统一，不利于后续模块的开发。

所以我们需要更好的方式来赋予前端轨迹时间参数。一个不那么简单但是直接的思路是将JPS轨迹作为KinoDynamic A* 的heuristic，通过JPS的全局性引导KinoDynamic A*。然而在实践中发现，这个方法的实现复杂于一次放松条件的Minco轨迹优化而且会引入额外的模块，增加系统复杂程度。所以我们提出了基于一次额外Minco轨迹优化的实现，新的实现流程是这样的:

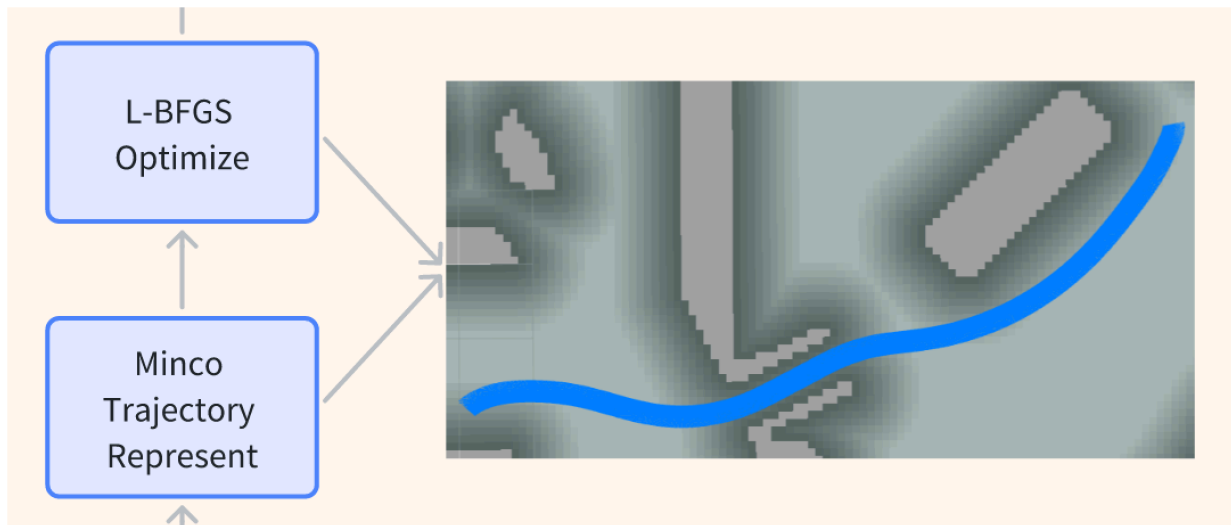
▲ 基于JPS的全局性引导KinoDynamic A*

- 放松时间条件和碰撞条件进行基于Minco轨迹的优化
- 基于思路一估计总时间并通过均匀时间差重采样Minco轨迹
- 整理得到最终的前端轨迹点和每个轨迹点的时间

这部分内容实际上已经和后续轨迹优化部分混合在了一起，不好说是叫后端的两次轨迹优化还是前端的一次宽松条件优化（。

5.5.4 MINCO轨迹表示和优化

Minco轨迹表示和相关工作可以参考[ZJU-FAST-Lab/GCOPTER: A General-Purpose Trajectory Optimizer for Multicopters](https://github.com/ZJU-FAST-Lab/GCOPTER: A General-Purpose Trajectory Optimizer for Multicopters)，也有很贴心的中文版本的论文[Chinese - A Geometrical Approach to Multicopter Motion Planning.pdf at main](#)。



GCOPTER仓库做了很好的实现，是目前全网最好的Minco使用案例（有意义废话）。如果感兴趣推荐仔细研究。

我们基于GCOPTER项目进行修改，修改到我们的版本：

- 修改到ROS2
- 替代安全走廊，基于ESDF提供避障梯度
- 替代无人机的三维微分平坦模型，修改为二维全向微分平坦模型（大大简化了）
- 设计基于ESDF和二维模型的梯度计算方式
- 添加两步优化
- 删除其它所有不需要的代码，精简项目

5.5.4.1 改进过程：

1. 发现**L-BFGS Optimization Failed: Linear Search Max**添加在优化失败后也输出结果，查看**轨迹形状已经基本符合要求**。失败原因不在于前端质量。分析认为原因来自于前面部分提到过的梯度无效化问题：**修改为二次插值，同时放大迭代次数上限**。仍然无法完全解决问题。参考DDR-OPT项目，查看它对于L-BFGS的修改：发现它添加了一个在 $\text{param.past} > 0 \ \&\& \ \text{fabs}(\text{finit}-f)/(\text{fabs}(\text{finit})+1.0) < \text{param.delta}/\text{param.past}$ 条件下的**退出单次优化条件**，添加以后基本可以保证优化成功
2. 发现**后端轨迹优化时间很长**：在1的解决基础上，一条30m长轨迹往往需要近100ms的优化时间。通过调参放松最终条件，同时记录迭代轮次，添加迭代轮次到达上限后的退出：成功控制优化时间在一个稳定值以内（单线程大概80ms）

3. **低速部分轨迹优化出问题**：在开始阶段附近存在转向将导致时间分配不合理（前端问题）。解决方案为前端中思路一种描述的**考虑转角的时间分配方案**。解决这一问题以后优化稳定性增强许多，但是仍然有几率出现优化不稳定问题。给出的解决方案是**前端中的思路二**，它基本解决了这个问题。
4. 添加两步优化，两步优化目标不同，第一次优化得到稳定优秀的轨迹形状，第二次优化得到良好的动力学特征。优化整体时间大大缩短，轨迹质量也获得了很大提升。

5.5.4.2 优化目标构建

这是需要特别讨论一下的部分，这个优化策略很大程度上提高了轨迹的可用性。优化过程分为两种策略，PRE_OPTIMIZATION和与它相对的FINELY_OPTIMIZATION。

- PRE_OPTIMIZATION：梯度为直接通过二次插值得到的梯度，不考虑沿轨迹方向和垂直轨迹方向的差别。这样的策略很自然，但是在狭窄隧道的前后方，控制点会被推离隧道。这样会导致两个控制点之间的距离变大很多，导致错误的轨迹速度。这样的速度会需要额外的减速段来完成减速，最终导致轨迹在速度不可用的同时形状也基本不可用。
- FINELY_OPTIMIZATION：在这里我们考虑速度方向，一个自然的思路是**不产生沿速度方向的梯度**。
 - 第一步是将梯度减去沿轨迹方向（轨迹上某点的速度方向）的梯度分量得到实际使用的梯度gradPos。
 - 同时我们需要知道这个点处在什么样的位置，这样的点可不可以在梯度下被移动一个step。如果可以的话，我们需要将它的梯度放大到正常水准（normalize操作，因为考虑单障碍物，一个step最多让ESDF值增大一个step大小，故norm到1）。所以我们根据gradPos的方向采样一个step后的ESDF值，这个值和当前点的值一步计算梯度。我们看梯度大小的含义，如果梯度接近1，那说明这个方向可以自由移动，不会出现优化到另一端的问题。否则我们需要特殊处理violaPos（和软约束相关）。
 - 我们设置一个关于上一步计算出梯度的阈值，如果梯度小于阈值，则认为点出于将会优化到势谷的位置。我们对violaPos进行特殊处理。这个阈值取定了0.5，取一个期望下比较合理的值。前面提到了我们的差值方式为二次插值，梯度值其实反映了距离峰谷的距离 $scale * (\sqrt{gradPos.norm()})$ 。我们最终采用这个值作为violaPos的大小，采用gradPos作为梯度方向。

在与避障相关的项之外，我们还设置了和速度，加速度相关的项。他们说很简单的二次loss，具体实现为平方减阈值平方，一次求导以后只剩下两倍本身，比较简单。

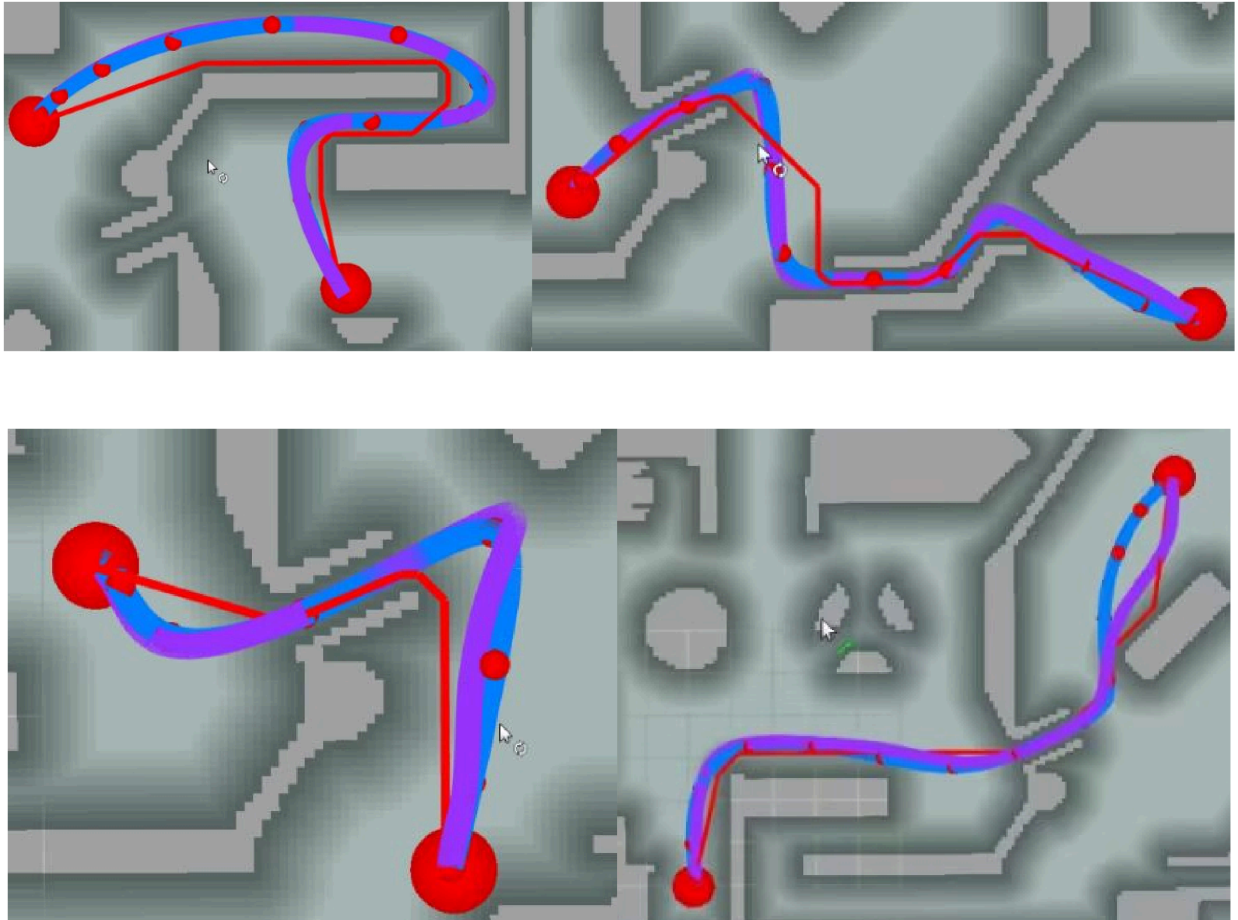
将轨迹上离散采样的三种梯度gradPos,gradVel,gradAcc汇总到轨迹后段的控制点（GradC GradT）上。

当然，我们还没有提第一次优化的时候是怎么避免控制点被推出狭窄区域的，我们的做法是添加一个根据时间约束。因为采样前端轨迹的假设是等时间间隔，所以我们希望第一次优化之后这个性质被基本保留。所以我们添加了段时间相对于轨迹的平均段时间的约束，通过二次约束进行构造。设置了上限和下限来进行控制，上限设置为1.1，下限取0.9。通过这样的操作得到关于时间的梯度累加到GradT上，获得cost加到最终的cost上。

5.5.4.3 单次规划效果

下图中的**紫色轨迹为一次优化轨迹，蓝色轨迹为二次优化轨迹**。从曲线和控制点可以看出蓝色轨迹的动力学性能明显更好（控制点之间基本为等时间间隔的），也更符合避障的要求。可是也存在一些要改进的地方，就是进入和狭窄区域缺少了“提前对齐”的效果（一次优化方案误打误撞实现了这个功能），这对控制器提出了更高要求。但是最好就是再想想怎么在规划模块更好解决这个问题。

同时因为这个方案相比于朴素的一次优化方案可以降低对控制点数目的要求，我们很大程度上减少了控制点数量，提高了轨迹优化速度。这两条轨迹的两次优化时间估计都在10ms以内。



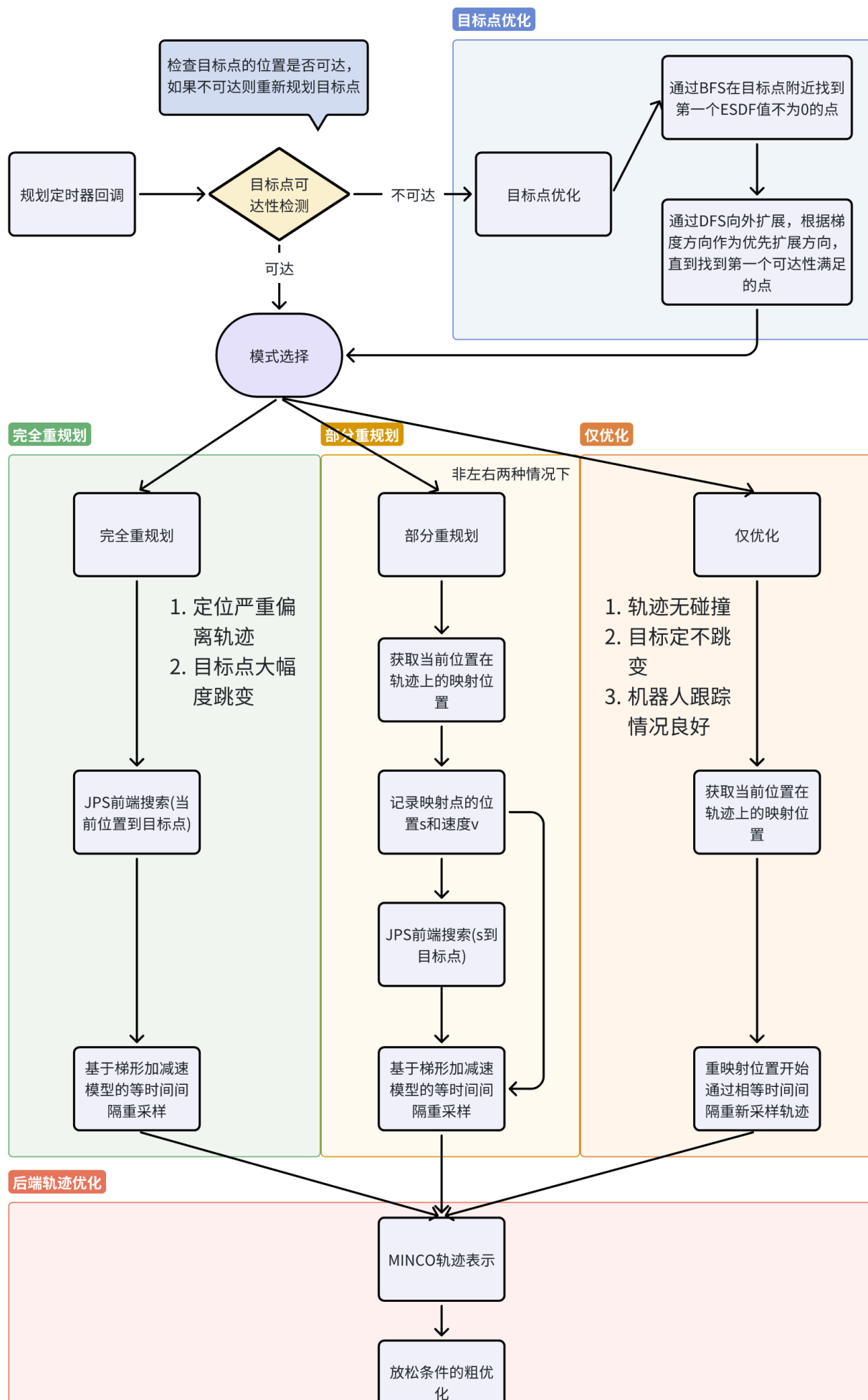
5.5.4.4 重规划策略

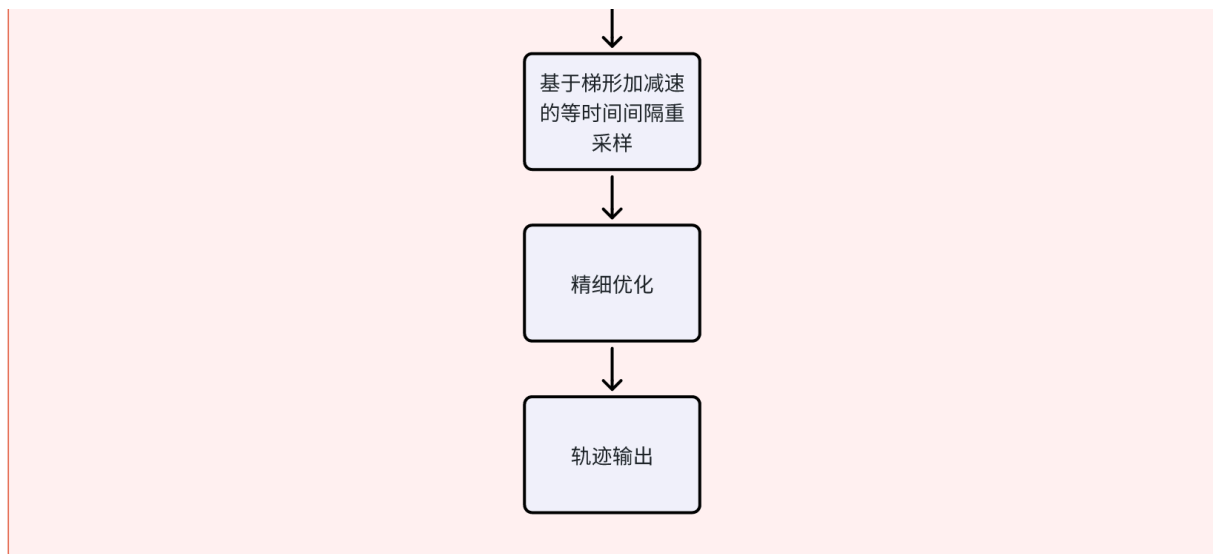
因为场景是高动态且多变的，重规划是系统能够安全运行的重要保证。

因为Minco通过路标点和时间对轨迹进行描述，而且轨迹必然经过路标点。这个特点给我们的重规划模块提供了很多的方便：

1. 局部重规划的起点是实时里程计位置在轨迹上的最近投影点。但是单单这样难以保证局部重规划后轨迹的稳定性。所以比较好的方式是保留已经走过的一小段轨迹在新轨迹的前方，这样能保证新的轨迹和老的轨迹在当前机器人位置上差别不大。如何实现这个功能则是在最近投影点上往回找一段时间，保留这部分轨迹。将它拼接到JPS搜索结果之前，然后送给优化器做优化。这样一来重新规划的的状态在机器人当前位置上基本没有跳变。
2. Minco经过路标的特性使得我们可以通过路标点，初始速度和每个路标点对应的时间戳在不同模块之间传递轨迹。后一环节只需要利用这些信息重新轨迹就可以获得和规划模块完全相同的信息（完全重现）。
3. Minco轨迹经过路标，所以梯度最后汇总到了路标点梯度上，这使得我们可以使用L-BFGS这样的无约束优化器高效简单地通过操作路标点控制轨迹形状和运动学特征。

重规划模块根据MINCO特性进行设计，高度契合轨迹表示策略。流程上大体如下图所示（通过流程图比文字易于理解得多）：





5.5.5 控制器设计

5.5.5.1 控制目标

哨兵的控制器主要是实现对规划器输出轨迹的跟踪，保证机器人在规划器输出的轨迹上行驶。我们需要考虑到哨兵的运动学和动力学约束，以及系统不同频率模块协作的延迟问题，以避免机器人出现打滑、翻车、震荡或迟滞的问题。

具体来说，我们主要考虑如下的条件与控制目标：

1. 控制器充分相信上游模块，即里程计、规划器等输出，即信任定位结果、轮速等输入等具有准确的时间戳，信任输入的规划轨迹无碰撞、连续。
2. 在此基础上，控制器需要考虑上下游模块的通信、计算和控制延迟，通过多传感器融合与命令前瞻降低延迟对于整体控制效果的影响。
3. 控制器的控制目标为：
 - 1) 输出速度控制命令，最小化机器人的实际轨迹与规划器输出的轨迹之间的误差
 - 2) 最大化机器人沿轨迹移动的速度
 - 3) 考虑机器人为全向底盘的运动学约束
 - 4) 满足机器人的动力学约束，体现为全向舵轮摩擦力约束

5.5.5.2 控制方案

基于以上的控制目标，我们前期进行了控制方案的对比分析，主要考虑了如下的控制器方案：

1. PID控制器
2. Pure Pursuit控制器
3. MPC控制器

在综合考虑系统延迟、轨迹前瞻能力与严格的约束施加要求后，我们对以下三种控制器方案进行了对比分析：

- PID控制器：

PID控制器结构简单，实现容易，在很多基础控制任务中能够取得较好的效果。但是在我们的设计场景中，其存在以下局限性：

- 难以考虑系统延迟与轨迹前瞻：PID控制器主要依赖当前和历史误差进行控制，无法充分利用规划器输出的未来轨迹状态来补偿下位机的延迟，容易导致振荡或响应滞后。

- 难以施加约束：在全向底盘的约束系统中（全向舵轮摩擦力约束），直接在PID控制器中引入这些约束较为困难，通常需要额外设计补偿机制，增加了系统调试和参数调整的复杂性。

- Pure Pursuit控制器：

Pure Pursuit控制器通过提前计算目标轨迹上的前瞻点，具备一定的轨迹追踪能力，其优点在于：

- 前瞻能力：能够在一定程度上利用未来轨迹点进行控制，使车辆提前调整姿态，降低响应延迟的影响。
- 运算简单、实时性好：适合实时在线跟踪，并能在计算上满足高频控制要求。

然而，针对我们的应用场景，Pure Pursuit也存在一些问题：

- 适用范围限制：该方法更适合作为自行车底盘或阿克曼底盘的简单跟踪方案，并不充分适合全向移动底盘的运动学特点。
- 难以施加严格约束：同样，Pure Pursuit在其控制算法中难以直接整合动力学约束，导致在某些极端工况下无法充分满足安全和稳定性要求。

- MPC控制器

模型预测控制（MPC）则通过构建未来一段时间的系统状态预测模型，并在此基础上解决一个带约束的优化问题，从而生成最优控制指令。在本设计中，我们将MPC转化为满足线性运动学约束的二次规划（QP）问题，从而具备以下优势：

- 充分补偿系统延迟与命令前瞻：MPC能够利用规划器的未来轨迹信息，有效补偿下位机PID跟随过程中造成的延迟，通过规划未来多个时刻的状态，实现控制性能的提前补偿。
- 易于约束处理：在QP问题中可以方便地加入运动学、动力学及摩擦等多种约束，保证控制行为在所有关键指标上都满足系统设计要求。
- 高效稳定与调参简便：由于问题被转化为具有良好求解特性的QP，调参数和调试过程相对简单，同时在实际场景中展现出高效、稳定的性能。

总结来说，MPC控制器（转化为线性运动学约束下的QP问题）能够充分利用规划器的未来轨迹信息，有效补偿计算与下位机计算的延迟，同时在加入运动学和动力学约束方面有着明显的优势，并且调参和实际调试过程中表现出高效、易调性。

因此，我们最终敲定了基于MPC控制器方案，作为哨兵控制器的最终设计方案。

5.5.5.3 MPC控制器设计

我们将详细介绍如何将MPC控制问题转化为标准的二次规划（Quadratic Programming, QP）问题，并使用开源的QP求解器qpOASES进行高效求解。

5.5.5.3.1 MPC问题的数学建模

我们首先回顾一下模型预测控制（MPC）的核心思想：

MPC通过优化未来一段预测窗口内的控制输入序列，最小化预测窗口内实际状态与参考轨迹的误差，同时考虑系统的运动学约束与动力学约束。

具体而言，我们以全向移动机器人为例，考虑线性离散化运动学模型：

$$x_{k+1} = Ax_k + Bu_k$$

其中：

- 状态量 $x_k = [p_x, p_y, \psi, v_x, v_y, \omega]^T$:
 - p_x, p_y : 机器人在二维平面中的 x, y 位置
 - ψ : 机器人在平面中的航向角（Yaw）
 - v_x, v_y : 机器人在 x 和 y 方向的速度
 - ω : 机器人绕 Z 轴的角速度（Yaw 角速度）

- **控制量** $u_k = [F_x, F_y, M_z]^T$:
 - F_x, F_y : 机器人在 x 和 y 方向的控制力
 - M_z : 机器人绕 Z 轴的控制力矩
- A, B 为系统的离散状态转移矩阵和控制输入矩阵

$$A = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{\Delta t}{m} & 0 & 0 \\ 0 & \frac{\Delta t}{m} & 0 \\ 0 & 0 & \frac{\Delta t}{I_z} \end{bmatrix}$$

其中:

- m : 机器人总质量
- I_z : 机器人绕 Z 轴的转动惯量
- Δt : 控制周期 (时间步长)

在预测窗口长度为 N 的MPC问题中, 我们定义状态预测序列 $X = [x_1^T, x_2^T, \dots, x_N^T]^T$ 和控制序列 $U = [u_0^T, u_1^T, \dots, u_{N-1}^T]^T$, 则系统预测模型可以表示为:

$$X = \bar{A}x_0 + \bar{B}U$$

其中:

$$\bar{A} = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}, \quad \bar{B} = \begin{bmatrix} B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \dots & B \end{bmatrix}$$

5.5.5.3.2 MPC优化问题的标准形式

考虑目标函数为跟踪参考轨迹的误差最小化和控制输入平滑性的综合代价, MPC优化问题通常表示为:

$$\min_U (X - X_{\text{ref}})^T Q (X - X_{\text{ref}}) + U^T R U$$

其中:

- X_{ref} : 参考轨迹序列
- $Q = \text{diag}(q_{p_x}, q_{p_y}, q_{\psi}, q_{v_x}, q_{v_y}, q_{\omega}) \succeq 0$: 状态误差权重矩阵 (通常为正半定对角矩阵)
- $R = \text{diag}(r_{F_x}, r_{F_y}, r_{M_z}) \succ 0$: 控制输入权重矩阵 (通常为正定对角矩阵)

带入系统预测模型 $X = \bar{A}x_0 + \bar{B}U$, 我们得到:

$$\min_U (\bar{A}x_0 + \bar{B}U - X_{\text{ref}})^T Q (\bar{A}x_0 + \bar{B}U - X_{\text{ref}}) + U^T R U$$

展开并整理, 我们便得到标准二次规划 (QP) 形式:

$$\min_U \frac{1}{2} U^T H U + f^T U$$

其中:

- Hessian矩阵 (H) 和Gradient向量 (f) 为:

$$H = 2(\bar{B}^T Q \bar{B} + R), \quad f = 2\bar{B}^T Q (\bar{A}x_0 - X_{\text{ref}})$$

5.5.5.3.3 约束条件的处理与转化

我们需要加入运动学与动力学约束，例如全向舵轮的摩擦力约束、车体速度约束以及控制输入约束。一般形式可表示为线性不等式约束：

$$U_{\min} \leq U \leq U_{\max}, \quad X_{\min} \leq X \leq X_{\max}$$

通过预测模型代入 $X = \bar{A}x_0 + \bar{B}U$ ，状态约束可转化为：

$$X_{\min} - \bar{A}x_0 \leq \bar{B}U \leq X_{\max} - \bar{A}x_0$$

与控制约束合并后，构成统一的线性不等式约束：

$$GU \leq g$$

其中 G, g 可由上述上下限矩阵直接构造。

在我们的具体问题中，考虑摩擦力约束：

$$|F_x| \leq F_{\max}, \quad |F_y| \leq F_{\max}, \quad |M_z| \leq M_{\max}$$

我们可以将其转化为线性不等式约束：

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} F_x \\ F_y \\ M_z \end{bmatrix} \leq \begin{bmatrix} F_{\max} \\ F_{\max} \\ M_{\max} \\ -F_{\max} \\ -F_{\max} \\ -M_{\max} \end{bmatrix}$$

5.5.5.3.4 转化为稠密QP问题与求解

综上所述，我们的MPC优化问题可统一表示为标准的QP形式：

$$\begin{aligned} \min_U \quad & \frac{1}{2}U^T H U + f^T U \\ \text{s.t.} \quad & GU \leq g \end{aligned}$$

该形式即为稠密二次规划问题（Dense QP），我们可直接利用开源QP求解器qpOASES进行求解。

5.5.5.4 控制器实现

我们在哨兵的控制器中实现了上述的MPC控制器设计方案，主要包括以下几个步骤：

1. 定位同步与估计
2. 首次期望轨迹生成与QP问题求解
3. 二次期望轨迹生成与QP问题求解
4. 控制命令输出

5.5.5.4.1 定位同步与估计

我们首先需要将里程计、轮速等输入数据进行同步与估计，确保控制器能够获得准确的状态信息。我们使用ROS2的时间戳机制来实现数据的同步，并在控制器中进行状态估计。

在具体实现中，上游Point-Lio里程计以100Hz工作，IMU以200Hz发布数据，轮速以1000Hz发布。我们在控制器中使用ROS2的时间戳机制来同步这些数据，并基于有延迟的里程计位姿借助高频轮速与IMU数据进行局部积分以估计最新的位置 p_x, p_y 与角度 ψ 。

5.5.5.4.2 首次期望轨迹生成与QP问题求解

由于轨迹规划器的输出频率较低，我们需要在控制器中进行轨迹起点的实时选取与前瞻期望轨迹生成。基于同步后的最新位置估计，我们计算出距离当前位置最近的Minco轨迹段，在其上投影自身位置作为轨迹起点，得到起始时间点 t_{start} ，并向后以 Δt 为步长选取轨迹点，直到达到规划器输出的轨迹长度或达到最大前瞻时间。

在期望轨迹的生成上， x, y 位置和速度期望状态直接由Minco轨迹生成，航向角则由速度切线方向得到。

即，对于 N 个离散点构成的期望轨迹上，第 i 个点， $0 \leq i < N$ ：

$$\begin{aligned} [p_{x,\text{ref},i}, p_{y,\text{ref},i}]^T &= \text{getPos}(t_{\text{start}} + \Delta t \cdot i) \\ [v_{x,\text{ref},i}, v_{y,\text{ref},i}]^T &= \text{getVel}(t_{\text{start}} + \Delta t \cdot i) \\ \psi_{\text{ref},i} &= \text{atan2}(v_{y,\text{ref},i}, v_{x,\text{ref},i}) \end{aligned}$$

进而，我们在控制器中实现了QP问题的构造与求解，主要包括以下几个步骤：

1. 初始化阶段，设置QP求解器的参数与约束条件，包括质量、转动惯量、状态权重、控制输入权重等。
2. 预构造QP问题的Hessian矩阵与约束不等式（由于我们的简化模型是时不变的简单线性模型，这些矩阵是静态的，预构造可以避免重复计算的时间浪费）。
3. 估计最新状态，获取参考轨迹。
4. 构造QP问题的Gradient向量。
5. 调用qpOASES求解器进行QP问题的求解，获取最优控制输入序列。得益于较低的QP问题维度，qpOASES求解器能够在较短的时间内（当预测步数为40步时，平均1ms内）完成求解。

这样我们就求解出了首次规划路径序列 $X_{\text{prim}} = [x_{\text{prim},1}^T, x_{\text{prim},2}^T, \dots, x_{\text{prim},N}^T]^T$ 和控制序列 $U_{\text{prim}} = [u_{\text{prim},0}^T, u_{\text{prim},1}^T, \dots, u_{\text{prim},N-1}^T]^T$

5.5.5.4.3 二次期望轨迹选取与QP问题求解

轨迹跟随中的跟随误差可以解耦为纵向跟随误差 e_{along} 和横向跟随误差 e_{cross} ，其中我们更关心后者，因为其决定了我们在穿过狭窄隧洞时会不会卡住。

我们在实践中发现，由于轨迹规划器更新频率较低，首次轨迹规划投影位置处的规划状态（位置、速度）与机器人实际状态可能存在较大差异，在笛卡尔系跟随一条起始状态与当前状态差异较大的轨迹时，若 x, y 方向误差权重相同则 e_{along} 容易“淹没” e_{cross} ，最终导致较大的 e_{cross} 。

为了解决这一问题，一种传统的方式是采用Frenet坐标系，但其在大曲率路线下存在线性化误差大和奇异点的问题，并且需要反复重新构建Hessian矩阵等，我们并未采取。

我们提出了一种极简且稳定的策略，根据首次规划的轨迹与期望轨迹上对应时间点的速度法向夹角余弦缩小时间步长重新选取轨迹点，从而在 e_{cross} 较大时通过主动“缩减”参考轨迹牺牲一定的纵向跟随速度来保证横向跟随精度。

即，对于 N 个离散点构成的二次期望轨迹上，第 i 个点， $0 \leq i < N$ ：

$$\begin{aligned} \alpha_i &= \max(0, \frac{v_{\text{prim},i}^T \cdot v_{\text{ref},i}}{|v_{\text{prim},i}| |v_{\text{ref},i}|}) \quad \text{where } v_{\text{prim},i} = [v_{x,\text{prim},i}, v_{y,\text{prim},i}]^T, v_{\text{ref},i} = [v_{x,\text{ref},i}, v_{y,\text{ref},i}]^T \\ [p_{x,\text{ref},i}, p_{y,\text{ref},i}]^T &= \text{getPos}(t_{\text{start}} + \sum_{j=0}^i \Delta t \cdot \alpha_j) \\ [v_{x,\text{ref},i}, v_{y,\text{ref},i}]^T &= \text{getVel}(t_{\text{start}} + \sum_{j=0}^i \Delta t \cdot \alpha_j) \\ \psi_{\text{ref},i} &= \text{atan2}(v_{y,\text{ref},i}, v_{x,\text{ref},i}) \end{aligned}$$

此后QP问题求解同（2）

5.5.5.4.4 控制命令输出

在 x, y 速度控制上我们将QP求解器输出的规划轨迹进行后处理，选取距离起始时间10ms的时间点的规划速度作为控制器的输出速度命令，发送给下位机进行执行。这是为了弥补MPC求解器计算的延迟、系统通信延迟与下位机PID跟随延迟的命令前瞻处理。

在旋转控制上，我们存在小陀螺前进与地盘跟随轨迹切向两种状态，根据环境的狭窄程度自动切换。在即将进入狭窄环境前，我们控制激光雷达所在的底盘前方旋转至沿轨迹切向正方向，而在进入狭窄环境后则就近使底盘旋转至轨迹切向正方向与反方向，以避免在狭缝中卡住。环境狭窄程度则由Minco轨迹控制点在esdf地图距障碍物的距离判断。

5.5.5.5 总结

总结来说，我们针对哨兵机器人的控制需求，设计并实现了一套基于MPC的控制方案。通过对控制目标的明确定义和多种控制器方案的对比分析，我们确定了采用MPC方法作为最终控制方案。相比于传统的PID控制器和Pure Pursuit控制器，MPC控制器能够更好地处理系统延迟问题，提供 stronger 的轨迹前瞻能力，同时能够更方便地引入运动学和动力学约束。

在MPC控制器的具体设计中，我们构建了全向移动机器人的线性离散化运动学模型，通过将MPC问题转化为标准二次规划形式，并借助开源求解器qpOASES进行高效求解。在具体实现过程中，我们特别关注了数据同步、轨迹二次以及命令输出前瞻等关键环节中对系统延迟的补偿，实现了流畅、准确且稳定的跟随控制。

5.5.5.6 讨论

出于简化调试难度，我们并没有直接将MPC规划的控制指令发送给下位机作为力控前馈，这在轨迹起点从0开始加速时造成了问题（前瞻的速度指令速度大小过小，下位机pid无法输出足够起步的推力）。我们的变通方法是在轨迹起点处自身低速时设置速度指令下限，但降低了低速时轨迹跟随的精度，或许是场上若干次卡在隧洞入口处的原因。MPC的最佳实践仍应使用规划的推力指令作为前馈以避免这一问题，但这也将带来额外的调试难度以解决震荡等问题。

5.5.6 总结

在这赛季我们从定位到感知到规划再到最后的决策提高了每一个模块的性能，应该算是突破了吧（笑）。

框架简单可靠，开发容易，debug方便，为我们节约了很多开发时间。相信这个框架已经是适用于轮式机器人的最优解决方案，我们将其完全开源并附上尽可能详细的文档，希望能为RoboMaster社区和产业界做出一些贡献。

我们会在线上视频平台公布这些模块的测试视频。

6 自瞄系统

现役的RW Vision的主要部分于24赛季联盟赛之后开始开发，基于RM Vision项目进行修改，在此对该项目作者表示感谢。

RW Vision开发于24赛季结束时停止，它在联盟赛中表现不错，但在分区赛更高强度的地面对抗下表现不甚理想。这让我们意识到了自瞄系统整体表现不取决于单一模块的“精益求精”，而在于木桶效应的最差短板。于是我们在RMUC2025分区赛到国赛时间内重点从以下几个方面解决系统瓶颈，提升了自瞄系统的整体表现：

- 高性能IMU模块
- 全自由度IMU相机标定
- 基于Bayer图像的灯条角点提取
- 对齐观测器的pnp和yaw refine
- 对侧对装甲板下凹灯条边缘被装甲板表面遮挡导致的灯条观测长度变化的建模与处理
- 更自然的观测器噪声表述

- 优化的IMU相机时间同步策略
- 高性能MPC云台控制器，开火控制器

由于时间和人力紧张，开发一直持续到了国赛期间，在深圳调试“大彻大悟”的部分：

- 识别基地
- 修正灯条长度与间距参考值
- 考虑枪管摆动对弹道的影响（易被忽略，但跟随高速旋转目标时有可观测的影响）

我们这段时间的开发留下了许多Bitter Lesson，也悟出了一些经验：

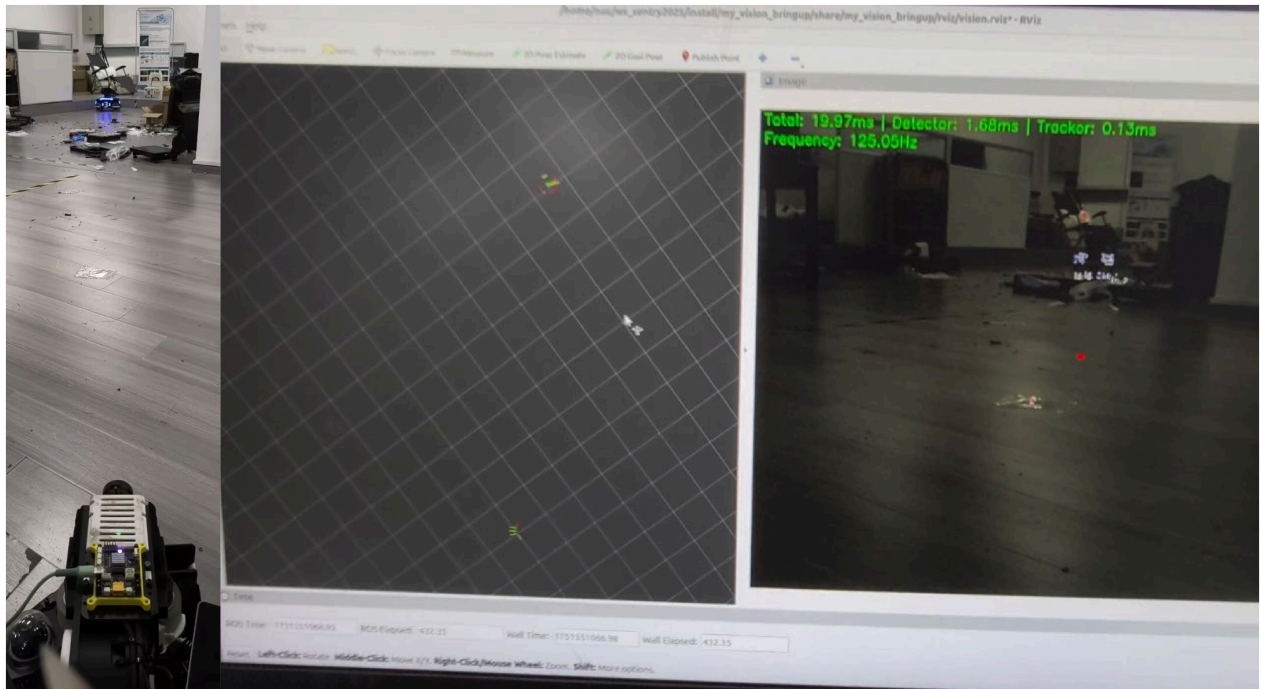
- 硬件和对硬件的标定至关重要：我们国赛全队普及了自己设计的高性能陀螺仪，这使得高质量的观测（敌方整车观测和自身云台角度、角速度）和MPC的部署成果可能。我们也非常重视陀螺仪Bias与Latency的标定，每天晚上标定效果的好坏直接影响第二天的上场表现
- MPC路线正确：我们分区赛时意识到一年没调的云台PID参数满足不了对高速小陀螺的跟随需求，临时报佛脚调整了几天参数没有本质的提升，这让赛后我们决心切换到MPC云台控制的路线。MPC将控制问题构建为优化问题，面对高速小陀螺目标的三角波输入可以从容规划整体轨迹，彻底解决了我们之前云台控制又超调又滞后的问题
- MPC调试难度较大：MPC直接力控云台是一个难以驾驭的方案，我们最终将规划的加速度、速度、位置同时下发给下位机，在双环PID基础上加入力矩、速度前馈，调试若干天才找到能用的参数，而且前瞻时间等关键参数稍有修改系统便开始震荡，缺乏经验的队伍建议谨慎尝试。相比之下，同济大学近期开源的轨迹规划方案应该更易于调试
- MPC的上限取决于机械：我们将同样的代码部署于步兵和哨兵后，哨兵做到了几乎零延迟跟随，而步兵存在10ms左右的跟随滞后。我们认为这是减速电机+同步带的力透明性不同导致的
- 关心小电脑：为什么步兵命中率不高是给NUC做局了，服役快三年的NUC在振动后会自己关机。最后两个比赛日发现问题并更换了NUC得到解决，大家记得平时保养NUC（悲）

我们的哨兵在场上以18发/s的弹频进行工作，可以实现在目标18rad/s小陀螺的情况下以50%的开火时间开火。最终数据为综合命中率32.2%（最高），局均伤害1184（最高），拿下最高哨兵KDA 6/0/5，最高单局伤害2577点。数据不会说谎（乐）。

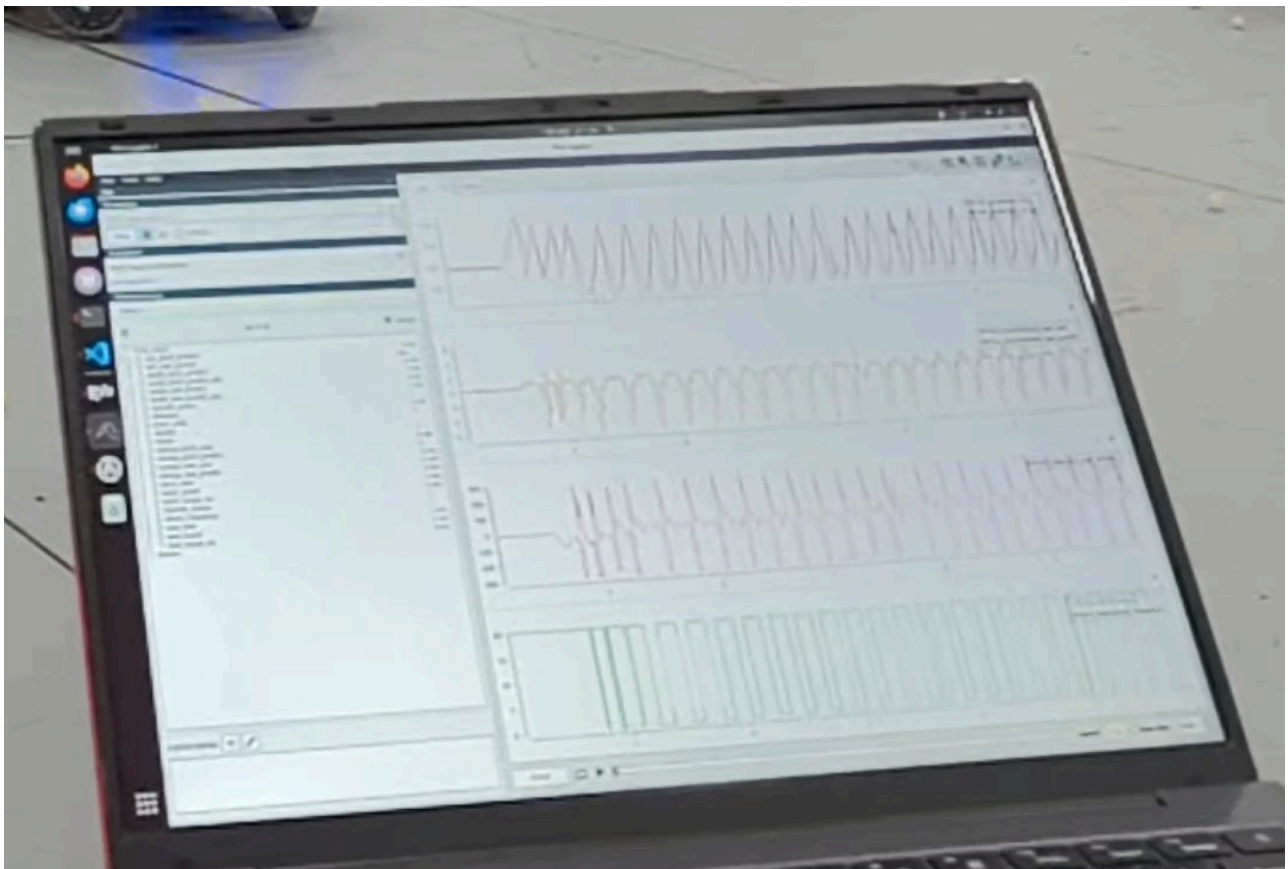
RW Vision2025只有一个整车观测器，没有对于平移靶的特殊处理，同时所有带自瞄机器人的自瞄都为RW Vision2025，仅参数差别。

我们在这里放一些测试时的样本：

10m左右，CS016-10UC相机，5mm镜头观测和识别，120w旋转+平移



下供弹步兵瞄准15rad/s目标，目标的位置，速度，加速度跟随曲线和开火建议（实在是找不到正常的图片了）

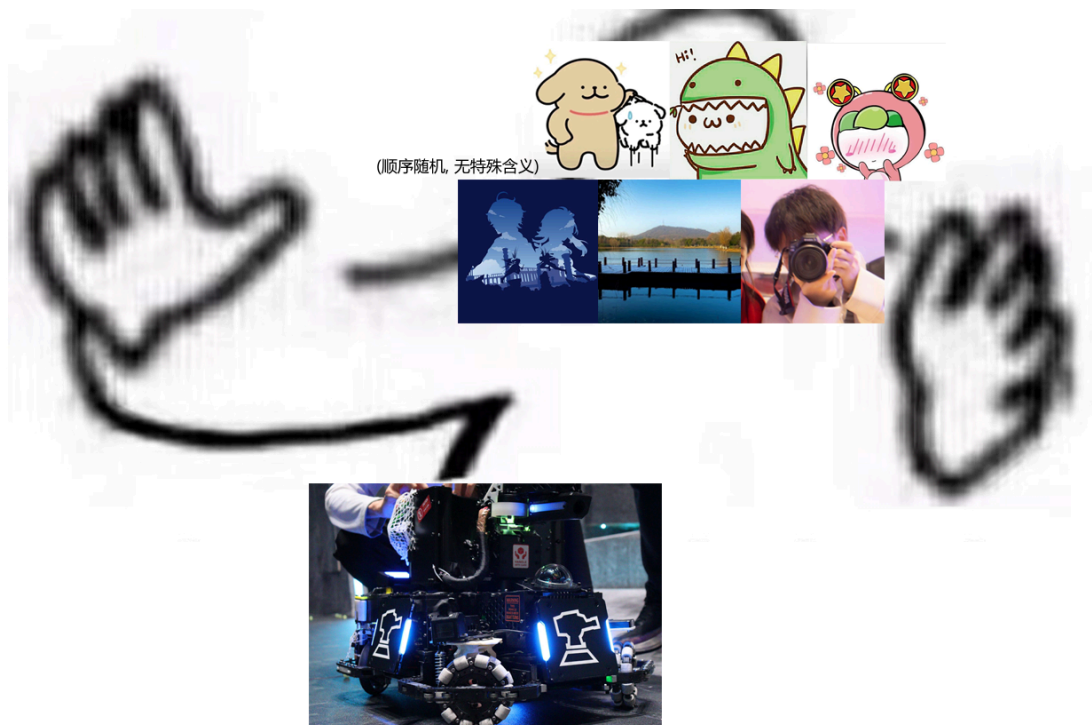


7 写在后面

综上所述我们希望今年的哨兵可以做到高效，稳定地满足战术目标。依靠更加高强度的控制和自瞄在比赛前期快速建立优势。跨越高地斩首对方英雄机器人拖缓对方吊射节奏

这是我们在赛季规划中写的。很高兴我们今年出色完成了这个目标。感谢各位的探索和付出。也很开心RoboWalker的每个人以良好的心态和状态，充满凝聚力的团队氛围完成了今年的比赛，这是今年最大的“技术突破”

这里泄露2025哨兵组合照：



哨兵自己打人!

8 参考与引用

markdown不好cite，故给出链接

[Point-LIO: Robust High-Bandwidth Lidar-Inertial Odometry](#)

[FAST LIO: A computationally efficient and robust LiDAR-inertial odometry \(LIO\) package](#)

[Faster-LIO: Lightweight Tightly Coupled Lidar-inertial Odometry using Parallel Sparse Incremental Voxels](#)

[ROG-Map: An Efficient Robocentric Occupancy Grid Map for Large-scene and High-resolution LiDAR-based Motion Planning](#)

[GCOPTER: A General-Purpose Trajectory Optimizer for Multicopters](#)

[Fast-Planner: A Robust and Efficient Trajectory Planner for Quadrotors](#)

[FIESTA: Fast Incremental Euclidean Distance Fields for Online Motion Planning of Aerial Robots](#)

[EGO-Planner-v2: Swarm Playground, the codebase of the paper "Swarm of micro flying robots in the wild"](#)

[DDR-opt: Universal Trajectory Optimization Framework for Differential Drive Robot Class](#)

[rm_vision: RoboMaster 视觉 ROS2 框架](#)

[Matrix and Robotics on STM32](#)

[【RM2024/25-哨兵机器人结构开源】-哈尔滨工业大学-I Hiter战队](#)